

JSS MAHAVIDYAPEETHA
JSS Science and Technology University



"Slient Bridge"

Event 1

Subject: Web Technology

Subject Code: 22CS620

Submitted by

Roll Number	USN	Name
23	01JST23UCS084	NEVISH SUVIN AMMANNA
28	01JST23UCS122	Rohan B
61	02JST23UCS099	Sarfaraj Ahamed
62	02JST23UCS101	Sheldon D'souza

Under the guidance of

Savitha N

Assistant Professor

Dept of Computer Science and Engineering

JSS Science and Technology University

Department of Computer Science and Engineering (2026-2027)

Table of Contents

Abstract.....	3
Core System Architecture and Technical Stack.....	3
Real-Time Translation and Intelligent Processing.....	3
Skeletal Three-Dimensional Visualization.....	3
Introduction.....	5
Project Vision and Objectives	5
The Technical Architecture	5
Interactive 3D Avatar and Skeletal Animation.....	5
Multimodal Input and Intelligent Processing.....	6
Importance of Silent Bridge	7
Implementation Guide for SilentBridge: An Assistive Web Application.....	7
Architectural Layout and Component Design	7
Core Rendering Engine and 3D Avatar Skeletal System	7
Speech Recognition and Linguistic Parsing Pipeline.....	8
Interactive Sign Verification with TensorFlow.js	9
Transformative Use Cases of SilentBridge.....	11
Facilitating Real-Time Communication and Accessibility	11
Educational Empowerment and Skill Development	11
Specialized Professional and Public Service Integration.....	11
Technical and Operational Challenges of SilentBridge.....	13
Real-Time 3D Rendering and Avatar Control.....	13
Linguistic Mapping and Sign Synthesis.....	13
Client-Side Machine Learning and Resource Constraints.....	13
Speech Recognition and Audio Processing Hurdles	14
State Management and Component Synchronization	14
The Evolution of Inclusive Digital Communication: Future Directions for SilentBridge.....	15
Advancing Machine Learning for Sign Language Nuance	15
Immersive 3D Avatars and Human-Computer Interaction	15
Enhancing Accessibility via Seamless Multimodal Inputs	15
Data-Driven Pedagogical Modules.....	16
Scalability and Web Standard Integration.....	16
Project Synthesis and Architectural Overview of SilentBridge.....	17
Foundational Web Technology Stack	17
Advanced Real-Time Animation Engine	17
Multimodal Input and Speech Processing	17
Human-Centric Accessibility and Interactive Modules	17
References.....	19

Abstract

Communication barriers between the hearing-impaired community and the general public remain a critical challenge in education, employment, and daily social integration. This project introduces **SilentBridge**, an innovative frontend-based web application designed to translate text and spoken language into real-time **Indian Sign Language (ISL)** animations. Developed using a robust suite of modern web technologies, the application leverages **React.js** for its component-based architecture and **Bootstrap** to ensure a responsive, highly accessible user interface across diverse devices.

The core translation pipeline integrates the native browser **Speech Recognition API** for hands-free voice input, alongside lightweight processing models powered by **TensorFlow.js** to parse natural language structure. Visualizing these gestures is achieved through a dynamic, high-performance rendering engine built on **THREE.js**, which drives a customized **3D avatar** via a sophisticated skeletal animation system. By using advanced interpolation algorithms, **SilentBridge** generates fluid, natural movement transitions between individual signs, bypassing the storage constraints and visual jerkiness of traditional video-based databases. Ultimately, the system serves as a scalable, interactive learning and translation utility aimed at fostering inclusive, barrier-free communication.

Core System Architecture and Technical Stack

- **React.js Framework:** Serves as the structural backbone of the web application, managing state and rendering modular components efficiently.
- **Bootstrap Styling:** Implements a mobile-first, responsive grid layout that adapts to various screen dimensions, ensuring the interface is highly accessible.
- **React Router Navigation:** Enables fluid client-side routing, allowing users to move seamlessly between translation tools, settings, and tutorials without page reloads.

Real-Time Translation and Intelligent Processing

- **Speech-to-Sign Conversion:** Utilizes the built-in browser **Speech Recognition API** to capture voice inputs and parse them into textual data on the fly.
- **Natural Language Parsing:** Leverages client-side algorithms to break down phrases into standard **Indian Sign Language (ISL)** grammatical structures.
- **On-Device Machine Learning:** Utilizes **TensorFlow.js** to handle linguistic pattern mapping directly in the browser, reducing server dependency and latency.

Skeletal Three-Dimensional Visualization

- **THREE.js Animation Engine:** Controls a fully rendered **3D avatar** directly in the browser workspace without requiring third-party plugins.
- **Skeletal Rigging System:** Employs hierarchical bones and joints within the avatar to accurately mimic complex hand shapes, finger spelling, and upper body postures.
- **Smooth Procedural Transitions:** Implements keyframe blending and mathematical interpolation to smooth out the gaps between consecutive signs, creating a natural flow of communication.

Introduction

SilentBridge represents a significant leap forward in assistive web technology, designed to bridge the communication gap between the deaf and hard-of-hearing communities and the hearing world. By utilizing modern web development standards, the platform translates spoken and written communication into real-time visual sign language. The core architecture of **SilentBridge** is entirely frontend-driven, ensuring rapid processing, enhanced user privacy, and cross-platform accessibility without the need for heavy backend infrastructure.

Project Vision and Objectives

The primary objective of **SilentBridge** is to democratize accessibility tools by providing a free, highly responsive, and interactive portal for **Indian Sign Language (ISL)** translation.

- **Breaking Communication Barriers:** Facilitates seamless, bilateral communication in educational, professional, and daily social environments.
- **Empowering through Education:** Incorporates specialized training modules that help users learn ISL gestures through visual repetition and interactive feedback.
- **Client-Side Efficiency:** Minimizes latency and operational costs by executing resource-intensive tasks, such as 3D rendering and machine learning, directly inside the user's web browser.

The Technical Architecture

The architecture of **SilentBridge** is built upon a highly performant and modern frontend stack designed to handle intense graphical rendering and real-time data processing.

- **React.js:** Acts as the structural backbone of the web application, leveraging virtual DOM manipulation to deliver a responsive, component-based user interface.
- **Bootstrap:** Utilized to build a modern, mobile-first responsive layout, ensuring that the application remains fully functional and visually consistent across mobile phones, tablets, and desktop computers.
- **React Router:** Implemented to manage client-side navigation dynamically, enabling smooth, instantaneous page transitions without full-page reloads, which preserves the state of the 3D avatar animations.

Interactive 3D Avatar and Skeletal Animation

At the heart of **SilentBridge** is a sophisticated rendering engine that translates linguistic data into physical movements.

- **THREE.js Engine:** Powers the underlying **WebGL** viewport, rendering a realistic 3D humanoid avatar capable of complex physical gestures.
- **Skeletal Animation System:** Leverages a structured hierarchy of virtual bones and joints, mimicking human physiology.
- **Smooth Gesture Transitions:**
 - Implements keyframe interpolation algorithms to prevent abrupt, robotic transitions between signs.
 - Generates realistic hand shapes, finger spelling, and arm movements essential for the precise representation of **Indian Sign Language**.
 - Allows users to control camera perspectives, zoom levels, and playback speeds for a customized viewing experience.

Multimodal Input and Intelligent Processing

To ensure maximum usability, **SilentBridge** supports multiple input modalities, allowing users to interact with the system in the manner that best suits their needs.

- **Text-to-Sign Engine:** Parses written text input, processes grammar rules matching the syntax of **Indian Sign Language**, and maps the output to corresponding animations.
- **Speech Recognition API:** Integrates the native browser-level speech recognition capabilities to enable hands-free voice-to-sign conversion in real-time.
- **TensorFlow.js:** Built into the system's pipeline to handle client-side machine learning tasks.
 - Prepares the application for computer vision capabilities, allowing for future expansion into sign-to-text recognition via the user's webcam.
 - Ensures private data processing, as visual inputs are evaluated locally on the user's device rather than uploaded to external servers.

Implementation Guide for SilentBridge: An Assistive Web Application

Architectural Layout and Component Design

Developing **SilentBridge** as a college project requires a modular, frontend-first approach to ensure smooth execution directly within the user's web browser. The system is designed to coordinate user input, process speech or text data, map linguistic tokens to sign representations, and render real-time animations.

- **Component-Driven UI with React.js**
 - The entire application is structured using **React.js functional components** to manage the state of the system dynamically.
 - The main application state tracks the **current animation queue, user audio streams**, and active interactive modules.
 - **React Hooks** such as ``useState``, ``useEffect``, and ``useRef`` manage the lifecycle of the 3D canvas and incoming speech tokens.
- **Seamless Client-Side Routing**
 - **React Router** manages multi-page navigation without triggering page refreshes, maintaining a persistent state for the 3D avatar.
 - Separate routes are established for ``/home`` (the core translator interface), ``/learn`` (interactive ISL modules), and ``/about`` (documentation).
 - Lazy loading is implemented via **React.lazy** to optimize initial page load times by splitting code based on routes.
- **Responsive Interface Construction**
 - **Bootstrap** provides the CSS grid system, ensuring a fully responsive dashboard layouts across mobile, tablet, and desktop viewports.
 - Custom UI modules leverage **Bootstrap utility classes** for flexbox alignment, card designs for learning modules, and accessible form elements.
 - Accessible themes and high-contrast color schemes are integrated using **CSS Variables** to accommodate visually impaired users.

Core Rendering Engine and 3D Avatar Skeletal System

The core of the translation system lies in rendering a 3D avatar and manipulating its bones to mimic human gestures in real-time. This is achieved through browser-native graphics rendering.

- **THREE.js WebGL Initialization**

- A **WebGLRenderer** is initiated within a dedicated React component, rendering a configured scene container with cameras, ambient lighting, and directional lighting.
- The 3D viewport utilizes an **PerspectiveCamera** configured with a wide field of view to track avatar movements without perspective distortion.
- **OrbitControls** are integrated to allow users to rotate and zoom around the avatar to inspect complex signs from different angles.

- **Avatar Model Loading and Animation Rigging**

- The 3D humanoid avatar is created and exported as a rigged **GLTF/GLB file**, containing standard skeletal bones for the hands, arms, neck, and spine.
- The model is asynchronously imported into the scene using the **GLTFLoader** plugin.
- Once loaded, the engine accesses the model's **SkinnedMesh** and individual bone hierarchies to dynamically update rotation vectors.

- **Skeletal Interpolation and Smooth Transitions**

- To prevent robotic, jerky movements, animation sequences are managed using **Quaternion-based rotations** instead of Euler angles.
- **THREE.Quaternion.slerp** (spherical linear interpolation) is used to transition smoothly from one sign posture to the next.
- A **frame rendering loop** running at 60 FPS continually updates the bone positions based on the calculated physics and interpolation rates.

Speech Recognition and Linguistic Parsing Pipeline

Translating spoken words or typed text into cohesive **Indian Sign Language (ISL)** requires a pipeline that captures input, cleans it, and maps it to specific animation keys.

- **Hands-Free Speech Recognition API Integration**

- The application implements the browser's native **SpeechRecognition API** (specifically ``webkitSpeechRecognition`` for wide compatibility).
- The system is configured for **continuous recognition**, allowing users to speak naturally while generating a real-time transcript stream.

- Error-handling triggers are built to manage background noise and speech pauses without crashing the audio listener.
- **Natural Language Parsing and Tokenization**
 - Raw text strings are processed to remove filler words, punctuation, and capitalizations that do not carry semantic weight in sign language.
 - The string is split into individual **lexical tokens** representing verbs, nouns, and adjectives.
 - **Indian Sign Language grammar structure** (which generally follows a Subject-Object-Verb order rather than English's Subject-Verb-Object) is applied via a custom parser.
- **Dictionary Mapping and Animation Queuing**
 - A local or hosted **JSON dictionary database** maps words and phrases directly to specific skeletal animation files.
 - If a word is missing from the database, the system defaults to a **finger-spelling fallback mechanism**, spelling out the word letter-by-letter.
 - An **animation playback queue** sequentially processes the translated tokens, feeding the corresponding bone rotation matrices to the THREE.js controller.

Interactive Sign Verification with TensorFlow.js

To make SilentBridge an interactive learning platform, users must be able to perform signs in front of their webcam and receive real-time correctness feedback.

- **On-Device Machine Learning Execution**
 - **TensorFlow.js** is deployed directly within the browser, utilizing client-side **WebGL acceleration** to process video frames locally without cloud dependencies.
 - This local execution guarantees absolute privacy for the user while minimizing backend server costs.
- **Pose and Hand Landmark Estimation**
 - The system loads the pre-trained **MediaPipe Hands** model via TensorFlow.js to locate 21 distinct 3D landmarks on each hand.
 - Frame streams from the user's camera are captured using standard **HTML5 Canvas** structures and piped into the model at regular intervals.
 - Coordinate datasets (representing joint angles and finger extensions) are extracted from each frame.

Transformative Use Cases of SilentBridge

Facilitating Real-Time Communication and Accessibility

The primary objective of SilentBridge is to bridge the communication gap between the hearing-impaired community and the general public. By utilizing **TensorFlow.js** for machine learning processing and **THREE.js** for high-fidelity rendering, the application serves as a dynamic translator.

- **Instant Speech-to-Sign Conversion:** The integration of the **Speech Recognition API** allows users to speak into their microphones, with the system immediately translating the audio into accurate **Indian Sign Language (ISL)** gestures performed by a 3D avatar.
- **Text-to-Sign Synthesis:** Users can input text directly into the interface, which the system processes to trigger specific skeletal animations, ensuring that information is accessible even in environments where vocal communication is not possible.
- **Enhanced Social Inclusion:** This application lowers the barrier for daily interactions, allowing individuals who use ISL as their primary language to participate more fully in public services, retail, and social settings without requiring a human interpreter.

Educational Empowerment and Skill Development

SilentBridge functions as an advanced pedagogical tool designed to standardize and teach Indian Sign Language in an interactive, digital environment. The application's **responsive user interfaces** ensure that learners can access educational content across various devices.

- **Interactive Learning Modules:** The platform features structured lessons where users can practice signs while receiving immediate visual feedback from the 3D avatar, helping to reinforce muscle memory and sign accuracy.
- **Self-Paced Training:** Because the system is built on **React.js**, the user experience is highly fluid, allowing students to navigate through different difficulty levels, review specific vocabulary, and track their progress through a intuitive web dashboard.
- **Standardization of ISL:** By providing a consistent digital reference for signs, the application helps in promoting a unified understanding of ISL, which is crucial for students who may have limited access to formal sign language schools or specialized instructors.

Specialized Professional and Public Service Integration

Beyond personal use, SilentBridge has significant potential for deployment in professional environments where accessibility compliance and inclusivity are mission-critical.

- **Customer Service Kiosks:** Institutions such as banks, hospitals, and government offices can embed SilentBridge into their digital terminals, enabling employees or automated kiosks to communicate directly with ISL users through a 3D interface.
- **Remote Work and Virtual Collaboration:** By leveraging the lightweight nature of **React Router** and optimized web-based rendering, the tool can be integrated into web-based collaboration platforms, allowing for more inclusive virtual meetings where sign language interpretation is handled autonomously by the system.
- **Public Information Broadcasting:** The system can be utilized to provide real-time updates in public transport hubs or transit stations, ensuring that critical announcements and emergency information are rendered into ISL animations, thereby catering to a wider demographic of users.

Technical and Operational Challenges of SilentBridge

Real-Time 3D Rendering and Avatar Control

- **Skeletal animation overhead:** Rendering highly detailed 3D avatars in real-time requires substantial GPU resources, which can severely degrade performance on budget mobile devices and older web browsers.
- **Transition blending:** Creating fluid transitions between discrete Indian Sign Language (ISL) signs is incredibly complex, often resulting in robotic or unnatural movements if interpolation keys are not precisely calculated.
- **Asset optimization:** Balancing the high-fidelity aesthetic of a 3D model with fast initial loading times requires aggressive optimization of **glTF/GLB models** and texture files to prevent bandwidth bottlenecks.
- **Finger tracking precision:** Representing intricate hand shapes and finger spellings in ISL demands highly detailed skeletal rigs, increasing the mathematical calculations required per frame in **THREE.js**.

Linguistic Mapping and Sign Synthesis

- **Grammar translation barriers:** ISL does not align with the grammatical syntax of spoken English or Hindi. Developing algorithms to convert linear text into spatially-structured sign sequences is a major linguistic hurdle.
- **Non-manual features:** Crucial aspects of ISL, such as facial expressions, head tilts, and shoulder movements, are difficult to coordinate dynamically with hand gestures using pure frontend scripting.
- **Contextual ambiguity:** Homonyms and context-dependent words require the translation engine to understand semantic intent rather than relying on simple word-to-sign dictionary lookup.
- **Regional variations:** Indian Sign Language has multiple localized dialects, making the creation of a standardized, universally accepted gesture database highly challenging.

Client-Side Machine Learning and Resource Constraints

- **Hardware dependency:** Running machine learning tasks via **TensorFlow.js** relies entirely on client-side hardware, which leads to inconsistent frame rates and thermal throttling on lower-end devices.

- **Model optimization:** Compressing complex neural networks into lightweight, web-friendly formats without sacrificing gesture recognition and translation accuracy is a delicate engineering trade-off.
- **Memory management:** Managing the lifecycle of web-based tensors is prone to memory leaks, which can eventually cause browser tabs to crash during extended interactive learning sessions.
- **Execution latency:** Real-time interactivity demands sub-100ms latency, which is difficult to achieve when simultaneous processing of audio, text translation, and 3D rendering occurs on a single main browser thread.

Speech Recognition and Audio Processing Hurdles

- **Accent and dialect variance:** The integration of the **Speech Recognition API** often struggles with the diverse range of Indian accents, leading to inaccurate text transcriptions.
- **Ambient noise sensitivity:** Background noise in real-world environments frequently pollutes audio input, degrading the accuracy of the hands-free speech-to-sign feature.
- **Browser compatibility:** Native Web Speech APIs have inconsistent support and behavior across different browsers and operating systems, particularly on iOS devices.
- **Network dependency:** Many cloud-backed speech-to-text engines require a constant, high-speed internet connection, making the application less reliable in areas with poor network coverage.

State Management and Component Synchronization

- **Lifecycle synchronization:** Coordinating the asynchronous lifecycle of **React.js** state updates with the continuous render loop of **THREE.js** can lead to state desynchronization and visual glitches.
- **Responsive design constraints:** Ensuring that a dynamic 3D WebGL canvas resizes fluidly within a flexible **Bootstrap** grid across smartphones, tablets, and desktops requires complex viewport calculations.
- **Route persistence:** Maintaining the active state, loaded models, and current animation progress when navigating between different views using **React Router** requires sophisticated state-preservation techniques.
- **Event loop blocking:** Heavy computations, such as parsing speech inputs or calculating inverse kinematics for the 3D rig, can block the main JavaScript thread, causing temporary UI freezes.

The Evolution of Inclusive Digital Communication: Future Directions for SilentBridge

Advancing Machine Learning for Sign Language Nuance

The integration of **TensorFlow.js** currently enables basic translation, but future iterations will leverage **Deep Learning** models trained on diverse regional datasets. Moving beyond basic word-to-sign mapping, the system can evolve to understand **Grammatical Syntax** and non-manual markers, such as facial expressions and body posture, which are critical components of **Indian Sign Language (ISL)**.

- Implementing **Pose Estimation** models to provide real-time feedback to users, allowing the system to act as both a translator and an instructor.
- Transitioning to **Transformer-based architectures** to better handle context-heavy sentences and ambiguous phrasing in real-time.
- Utilizing **Edge Computing** to process complex neural network computations directly on the user's browser, reducing latency and reliance on cloud servers.

Immersive 3D Avatars and Human-Computer Interaction

While **THREE.js** provides a robust framework for rendering 3D environments, the next phase of development involves enhancing the realism and emotional intelligence of the virtual avatars. The goal is to move from robotic movements to fluid, human-like signing that captures the speed and intensity of natural language.

- Incorporating **Inverse Kinematics (IK)** to allow avatars to interact more dynamically with virtual objects or educational prompts.
- Exploring **WebGPU** integration to achieve higher fidelity skeletal animations, ensuring that the visual performance remains consistent across various device hardware.
- Enhancing **Visual Fidelity** through improved mesh skinning and character texturing, making the avatar's hands and fingers more distinct to improve the clarity of complex sign sequences.

Enhancing Accessibility via Seamless Multimodal Inputs

The reliance on **Speech Recognition API** is a strong starting point for hands-free interaction, but the future of **SilentBridge** lies in creating a truly bidirectional communication tool. By expanding input methods, the platform can bridge the gap for users with varying levels of motor or vocal abilities.

- Developing **Multimodal Input Pipelines** that combine computer vision for hand-tracking with traditional text input for comprehensive accessibility.
- Adopting **Progressive Web App (PWA)** standards to allow offline functionality, ensuring that users in regions with unstable internet connections can still access essential translation services.
- Improving **Browser-based Audio Processing** to filter background noise in real-world scenarios, making speech-to-sign conversion reliable in crowded or public environments.

Data-Driven Pedagogical Modules

The interactive learning component of the application will shift from static content delivery to **Personalized Adaptive Learning**. By analyzing user progress, the system can tailor the difficulty and pace of learning modules, ensuring a customized experience for every student.

- Integrating **Learning Analytics** dashboards to track user improvement over time, identifying which signs or grammatical structures require more practice.
- Creating **Gamification Features** powered by React state management to increase engagement and retention rates among younger demographics and educational institutions.
- Utilizing **Community-Crowdsourced Datasets** to incorporate regional variations and dialects, ensuring the platform remains inclusive of the diverse signing styles present across India.

Scalability and Web Standard Integration

As the web platform grows, maintaining a **Responsive User Interface** while managing heavy 3D assets will require a shift toward advanced optimization techniques. By leveraging the latest web standards, the system can ensure maximum reach across both mobile and desktop platforms.

- Implementing **Asset Compression and Lazy Loading** for 3D models to ensure the application remains lightweight and fast-loading.
- Adopting **WebAssembly (Wasm)** to port high-performance C++ or Rust-based physics engines for more complex, life-like avatar movements that browser-based JavaScript might struggle to render efficiently.
- Ensuring **W3C Accessibility Compliance** throughout the development lifecycle to make sure the platform itself is navigable for individuals with diverse physical impairments, beyond those who specifically require ISL translation.

Project Synthesis and Architectural Overview of SilentBridge

Foundational Web Technology Stack

- The frontend architecture of SilentBridge is anchored on **React.js**, leveraging its component-based design to deliver a highly modular and maintainable codebase.
- User interface layout and responsive styling are managed using **Bootstrap**, ensuring seamless cross-device compatibility across desktops, tablets, and mobile screens.
- Routing and application state navigation are controlled via **React Router**, facilitating smooth, uninterrupted page transitions without full-page reloads.
- Client-side intelligence is augmented using **TensorFlow.js**, enabling the browser to handle machine learning and recognition tasks directly without heavy backend dependencies.

Advanced Real-Time Animation Engine

- The core animation pipeline is driven by **THREE.js**, which manages a sophisticated **skeletal 3D animation system**.
- This graphics framework enables realistic rendering of a digital avatar that translates linguistic data into physical movements.
- Key frames are rendered in real time, focusing on smooth transitions to capture the nuance of **Indian Sign Language (ISL)** gestures.
- The rendering engine optimizes CPU and GPU usage to ensure that complex joints and mesh coordinates move with high fidelity and zero lag.

Multimodal Input and Speech Processing

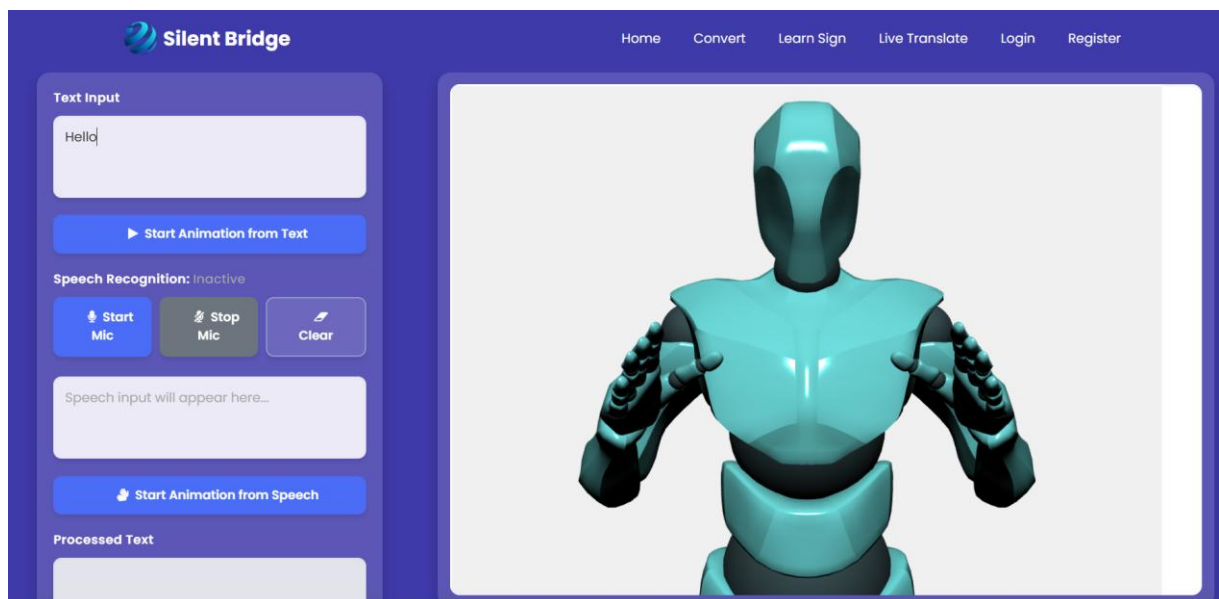
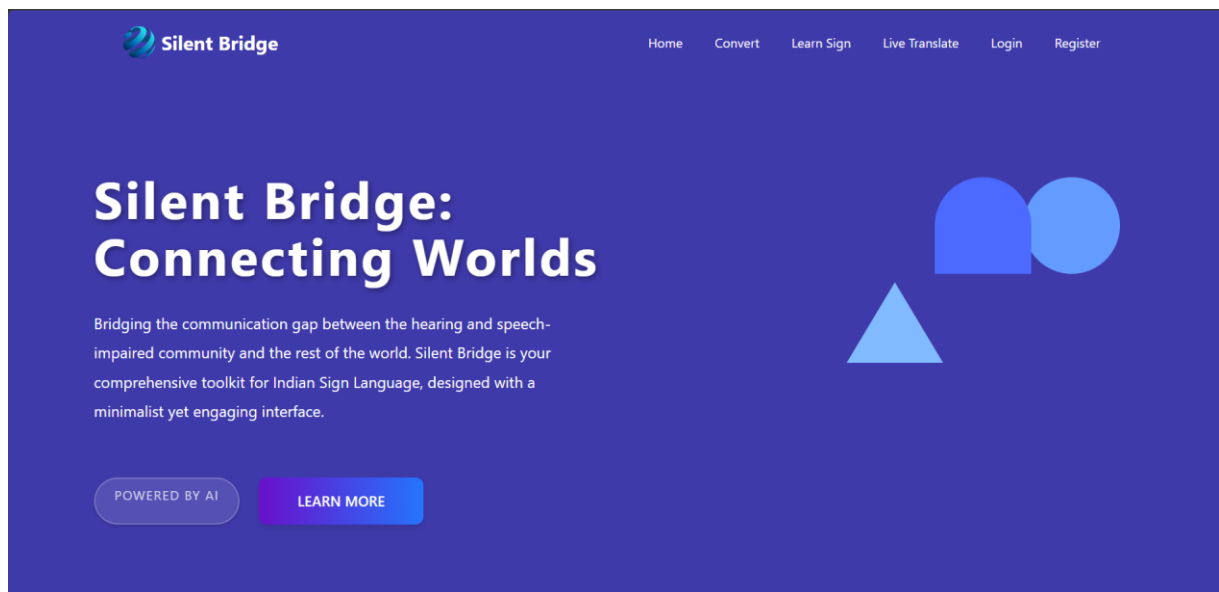
- The system incorporates the native browser-based **Speech Recognition API**, allowing users to speak directly into their devices.
- Voice inputs are captured and converted to text in real time, establishing a robust pipeline for hands-free **speech-to-sign conversion**.
- Alternative input methods allow users to type text directly, which is then parsed by the application's translation algorithms.
- Synthesized inputs are mapped dynamically to corresponding anatomical movements of the **3D avatar**, bridging the communication gap instantly.

Human-Centric Accessibility and Interactive Modules

- SilentBridge prioritizes digital inclusivity by offering tailored **interactive learning modules** designed to teach users the basics of Indian Sign Language.

- The UI features customizable accessibility settings to assist individuals with diverse learning paces and visual preferences.
- Client-side data processing minimizes latency, making the application highly dependable even in environments with limited internet bandwidth.
- By combining **modern web standards** with assistive technology, the platform empowers deaf and hard-of-hearing individuals, fostering broader social integration.

SNAP SHOTS



Silent Bridge

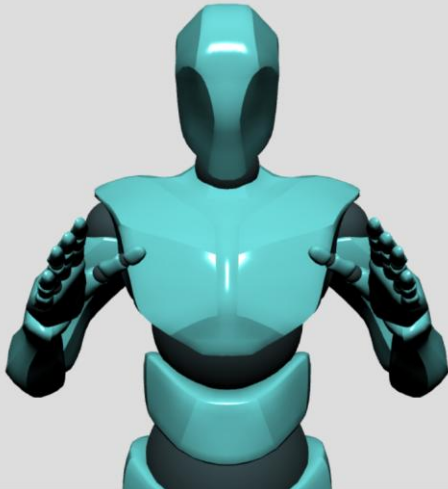
HomeConvertLearn SignLive TranslateLoginRegister

Alphabets

A	B	C	D
E	F	G	H
I	J	K	L
M	N	O	P
Q	R	S	T
U	V	W	X
Y	Z		

Words

TIME	HOME	PER...
YOU	HEL...	HOW



Silent Bridge Login

Email

Password

Login

Don't have an account? [Register here](#)

Silent Bridge Registration

Username

Email

Password

Role

User

Register

Already have an account? [Login here](#)

14. REFERENCES

- [1] Koller, O., Ney, H., & Bowden, R. (2015). *Deep Learning of Mouth Shapes for Sign Language*. International Conference on Computer Vision Workshops (ICCVW), 85-91.
- [2] Pigou, L., Dieleman, S., Kindermans, P., & Schrauwen, B. (2014). *Sign Language Recognition Using Convolutional Neural Networks*. European Conference on Computer Vision Workshops (ECCVW), 572-578.
- [3] Starner, T., Weaver, J., & Pentland, A. (1998). *Real-Time American Sign Language Recognition Using Desk and Wearable Computer Based Video*. IEEE Transactions on Pattern Analysis and Machine Intelligence, 20(12), 1371-1375.
- [4] Ong, S. C. W., & Ranganath, S. (2005). *Automatic Sign Language Analysis: A Survey and the Future Beyond Lexical Meaning*. IEEE Transactions on Pattern Analysis and Machine Intelligence, 27(6), 873-891.
- [5] Cooper, H., Bowden, R., Ong, E. J., & Pugeault, N. (2012). *Sign Language Recognition*. In Visual Analysis of Humans, Springer, 539-562.
- [6] Rastgoo, R., Kiani, K., & Escalera, S. (2021). *Sign Language Recognition: A Deep Survey*. Expert Systems with Applications, 164, 113794.
- [7] Bragg, D., et al. (2019). *Sign Language Recognition, Generation, and Translation: An Interdisciplinary Perspective*. ACM SIGACCESS Accessibility and Computing, 125, 1-12.